LAB 17 - PROMPTS

Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab17-tradingview-webhook/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Set the shared secret (A7)

Do this (no prompt needed)
In labs_fullcode/.env set WEBHOOK_SECRET to a long random string (for example the output of python3 -c "import secrets; print(secrets.token_urlsafe(24))"). Without it the receiver uses a demo secret and prints a warning.

Check: python -m pipeline check shows WEBHOOK_SECRET 'found (hidden)'.

Step 2: Build alert validation (A7, K5)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/webhook.py with class Gateway(cfg, session=None): secret from pipeline.config.secret(cfg, 'webhook_secret') (demo value 'lcc-demo-secret' if unset), broker = common.broker.get_broker(state_file='webhook_sim.json', reset=True), an OrderManager, a RiskManager and a set of processed alert ids. handle(payload) -> (http_code, body): 401 if hmac.compare_digest of the secret fails; ticker 'NASDAQ:AAPL' -> 'AAPL'; 422 if the ticker is not in risk.allowed_symbols, the action is not buy/sell, qty is not a number or not 0 < qty <= risk.max_order_qty; alert id = payload 'id' or a hash of ticker, action, qty and time; a repeated id returns 200 'duplicate ignored' (TradingView retries are safe). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: Gateway(load_config()).handle({'secret': 'guess'}) returns (401, ...).

Step 3: Route valid alerts through signal, risk and execution (A7, K7)

Prompt - copy into your AI coding assistant
In Gateway.handle add: pipeline_signal(ticker) = the configured strategy's latest target from pipeline.signals for that ticker; if webhook.require_signal_agreement is true and the alert disagrees, return 409; a sell with no position returns 422 (long only); RiskManager.check (kill switch, session, daily loss) failing returns 423 'blocked by risk'; otherwise target shares = held + qty for a buy or held - qty for a sell and call OrderManager.rebalance({ticker: target}, tag='tv<alert id>', units='shares') so the order is session-aware and idempotent; return 200 with order id, status, side, qty, fill price, pipeline_signal and signal_agrees. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: A valid buy returns 200 'order submitted' with pipeline_signal and signal_agrees in the body.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the server, the simulator and the self-test (A7)

Prompt - copy into your AI coding assistant
In labs_fullcode/pipeline/webhook.py add a standard-library http.server receiver (no Flask): POST webhook.path parses JSON (400 'body is not valid JSON' otherwise), calls Gateway.handle, appends a row to _outputs/Lab17_webhook_log.csv (time, client, alert_id, ticker, action, qty, http, result, pipeline_signal, order_id, status, detail; header written fresh when the server starts) and replies JSON; GET /health returns status, broker mode and kill switch. serve(cfg, host, port, max_requests) handles one request at a time; offline it uses a simulated regular session. test_alerts(secret) returns seven cases with expected codes: valid BUY 5 SPY (200), valid SELL 5 AMEX:SPY (200), wrong secret (401), TSLA (422), qty 5000 (422), malformed text (400), a retry of alert 1 (200 duplicate). send_test_alerts(url, secret) posts them with urllib and prints OK or MISMATCH; selftest(cfg) runs the server on a free port in a thread, sends the alerts and prints 'RESULT: n/7'. main(argv) for `python -m pipeline webhook [--port] [--max-requests] [--selftest]` and alert_sim_main(argv) for `python -m pipeline alert-sim --url --secret`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: python -m pipeline webhook --help shows --host, --port, --max-requests and --selftest.

Step 5: Run the self-test (A7, K5)

Do this (no prompt needed)
Run: python -m pipeline webhook --selftest.

Check: Seven alerts: 1 BUY 5 SPY -> 200 'order submitted', filled at 676.70, pipeline_signal 1, agrees; 2 SELL 5 SPY -> 200, filled at 676.57, signal_agrees false; 3 -> 401 'bad or missing secret'; 4 -> 422 'TSLA not on the allow-list'; 5 -> 422 'qty must be between 0 and 100'; 6 -> 400 'body is not valid JSON'; 7 -> 200 'duplicate ignored'; 'RESULT: 7/7 alerts handled as expected'; _outputs/Lab17_webhook_log.csv with 7 rows.

Step 6: Run the receiver and the simulator in two terminals (A7)

Do this (no prompt needed)
Terminal 1: python -m pipeline webhook --port 8080 (same as: python3 Lab17-tradingview-webhook/webhook_stage.py --port 8080). Open http://127.0.0.1:8080/health in a browser. Terminal 2: python3 Lab17-tradingview-webhook/send_test_alert.py --url http://127.0.0.1:8080/webhook. Stop the receiver with Ctrl+C.

Check: The health page shows status ok, broker offline-sim (or alpaca-paper) and kill_switch false; the simulator prints the same seven OK lines; the receiver prints each [code] and its stop summary.

Step 7: Let the pipeline's signal veto an alert (A7, K7)

Do this (no prompt needed)
In pipeline/config.yaml set webhook.require_signal_agreement: true and rerun python -m pipeline webhook --selftest. Set it back to false.

Check: Alert 2 (SELL while the pipeline's ema_cross signal for SPY is 1) now returns 409 'pipeline signal for SPY is 1: it disagrees with the alert' - RESULT 6/7, as designed.

Step 8: Prove the kill switch reaches the webhook (A7)

Do this (no prompt needed)
Run: python -m pipeline kill --on, then python -m pipeline webhook --selftest, then python -m pipeline kill --off.

Check: Alert 1 returns 423 'blocked by risk - KILL_SWITCH present - trading halted' and alert 2 returns 422 (no position to sell); RESULT 4/7 while halted.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 9: Optional - connect a real TradingView alert (paid plan) (A7, K5)

Prompt - copy into your AI coding assistant
Review labs_fullcode/Lab17-tradingview-webhook/tradingview_alert_message.json and lcc_webhook_strategy.pine. Explain which TradingView placeholders fill ticker, action, qty and id, why the id makes retries safe, and three ways to keep the shared secret from leaking (who can see alert messages, chart sharing, logs). Do not change the files.

Open in Claude | **Open in ChatGPT**

Check: Each real alert appears in the receiver console and in _outputs/Lab17_webhook_log.csv; online the paper order appears in the Alpaca dashboard.

Run it

Step	Command / file
Run Lab 17	python3 Lab17-tradingview-webhook/webhook_stage.py (same as: python -m pipeline webhook)
Also in this lab: send_test_alert.py	python3 Lab17-tradingview-webhook/send_test_alert.py
The complete code	pipeline/webhook.py (reference copy: Lab17-tradingview-webhook/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.